

Meta-Path Attention-Based Link Prediction in a Heterogeneous Biomedical Knowledge Graph

Introduction

The Dataset

In this project, we used the OGB BioKG dataset, a large-scale heterogeneous biomedical knowledge graph provided as part of the Open Graph Benchmark (OGB). The dataset integrates multiple types of biomedical entities and relations into a single graph structure, making it suitable for link prediction tasks in complex biological systems.

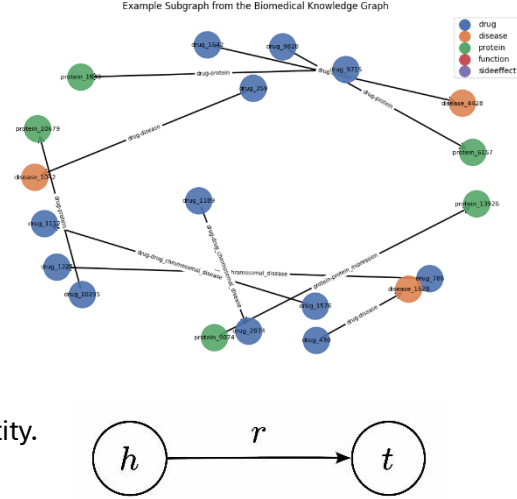
Unlike homogeneous graphs, where all nodes and edges are of the same type, BioKG is a heterogeneous graph composed of multiple entity categories and relation types. This structure enables the representation of diverse biomedical interactions such as drug–disease associations, protein functions, and side effects.

The dataset used in this project contains the following node types:

- Disease: 10,687 nodes
- Drug: 10,533 nodes
- Function: 45,085 nodes
- Protein: 17,499 nodes
- Side Effect: 9,969 nodes

Overall, the graph contains 51 different relation types connecting the various biomedical entities.

Each triplet in the graph is represented in the form: (h, r, t) . where h is the head entity, r is the relation type, and t is the tail entity.



Project Objective

The model scores a given biomedical triplet (h, r, t) and predicts whether this specific relation between the head and tail entities exists in the graph. The relation type is given as input and is not predicted by the model. In addition to standard node and relation embeddings, the project incorporates meta-path-based structural information and attention mechanisms in order to capture higher-order relationships within the graph and improve prediction performance.

Method

Embedding Initialization

At the beginning of the training process, each node and each relation is assigned a randomly initialized embedding vector. Since the graph is heterogeneous, separate embedding tables are maintained for different node types such as drugs, diseases, proteins, and functions. Each node v and relation r is therefore represented by a learnable vector: $H_{drug}^{(0)} \in \mathbb{R}^{N_{drug} \times d}$, $H_{Disease}^{(0)} \in \mathbb{R}^{N_{Disease} \times d}$, $H_{Protein}^{(0)} \in \mathbb{R}^{N_{Protein} \times d}$, $H_{Function}^{(0)} \in \mathbb{R}^{N_{Function} \times d}$, $H_{SideEffect}^{(0)} \in \mathbb{R}^{N_{SideEffect} \times d}$, $H_{Relation}^{(0)} \in \mathbb{R}^{N_{Relation} \times d}$, where $d = 128$ is the embedding dimension used throughout the project. This dimension was

chosen as a balance between representation capacity and computational efficiency: smaller dimensions may limit the ability to capture complex graph structure, while larger dimensions significantly increase memory usage and training time. During training, these embeddings are updated using backpropagation so that meaningful structural and semantic relationships are learned in the embedding space.

Meta-Path Construction and Encoding

After the initial embeddings are created, the model uses meta-paths to capture structural information beyond direct node-to-node connections. We selected meta-paths consisting of two edges

$$V_1 \xrightarrow{R_1} V_2 \xrightarrow{R_2} V_3$$

where: V_i denotes a node type and R_i denotes a relation type.

Meta-paths were selected based on their frequency in the graph. For each, the number of valid instances was counted. The meta-paths were then ranked by the number of instances, and a subset of frequent meta-paths was selected for the model. The two most frequent meta-paths were skipped, and the next 12 were selected, in order to avoid using only overly dominant patterns and less informative meta-path patterns.

For each selected meta-path type, 20 valid instances were sampled from the graph. Each instance consists of the three nodes with corresponding node embeddings:

$$\mathbf{h}_{start}^{(i)}, \mathbf{h}_{middle}^{(i)}, \mathbf{h}_{end}^{(i)} \in \mathbb{R}^{128}$$

The embeddings of the nodes participating in the same meta-path instance were concatenated to form a single path representation:

$$\mathbf{x}_i = [\mathbf{h}_{start}^{(i)}; \mathbf{h}_{middle}^{(i)}; \mathbf{h}_{end}^{(i)}]$$

where: $\mathbf{x}_i \in \mathbb{R}^{384}$. The concatenated vector was then passed through a learnable path encoder consisting of a linear transformation followed by a hyperbolic tangent activation function:

$$\mathbf{p}_i = \tanh(W\mathbf{x}_i + b)$$

where: $W \in \mathbb{R}^{128 \times 384}$. The resulting vector $\mathbf{p}_i$ represents the embedding of the i -th meta-path instance.

Intra-Meta-Path Attention

To estimate the importance of different meta-path instances, an attention mechanism was applied.

Alignment: $s_i = \mathbf{a}^T \mathbf{p}_i$, where $\mathbf{a} \in \mathbb{R}^{128}$ is a learnable attention vector.

Attention Weights: $\alpha_i = \frac{\exp(s_i)}{\sum_j \exp(s_j)}$ such that: $\sum_i \alpha_i = 1$

Context Vector $\mathbf{m} = \sum_i \alpha_i \mathbf{p}_i$ where $\mathbf{m} \in \mathbb{R}^{128}$ is the aggregated embedding of the meta-path type.

Inter-Meta-Path Attention

After obtaining an embedding vector for each meta-path type, an inter-meta-path attention mechanism was applied in order to estimate the relative importance of different meta-path types. In the current implementation, the meta-path representation is global and shared across all nodes, rather than being computed individually for each

target node as in the original MAGNN architecture. This simplification reduces computational complexity. Let: $\mathbf{m}_1, \mathbf{m}_2, \dots, \mathbf{m}_K$ denote the embeddings of the K selected meta-path types, where: $\mathbf{m}_i \in \mathbb{R}^{128}$.

Alignment: $s_i = \mathbf{q}^T \mathbf{m}_i$ where: $\mathbf{q} \in \mathbb{R}^{128}$ is a learnable attention vector.

Attention Weights: $\beta_i = \frac{\exp(s_i)}{\sum_j \exp(s_j)}$ such that: $\sum_i \beta_i = 1$.

Context Vector: $\mathbf{m}_{final} = \sum_i \beta_i \mathbf{m}_i$ where: $\mathbf{m}_{final} \in \mathbb{R}^{128}$ is the final aggregated meta-path representation used in the downstream link prediction task.

Before training, the attention weights were approximately uniform, as expected: $\frac{1}{12} \approx 0.083$ since 12 meta-path types were used in the model.

0.0836		protein	--r50-->	protein	--r50-->	protein
0.0803		protein	--r46-->	protein	--r46-->	protein
0.0846		protein	--r46-->	protein	--r50-->	protein
0.0775		protein	--r50-->	protein	--r46-->	protein
0.0817		protein	--r45-->	protein	--r45-->	protein
0.0677		protein	--r45-->	protein	--r50-->	protein
0.0844		protein	--r50-->	protein	--r45-->	protein
0.1016		drug	--r6-->	drug	--r41-->	sideeffect
0.0693		protein	--r45-->	protein	--r46-->	protein
0.0832		protein	--r46-->	protein	--r45-->	protein
0.0853		drug	--r13-->	drug	--r41-->	sideeffect
0.1008		drug	--r6-->	drug	--r6-->	drug

Link Prediction Head

The final stage of the model performs link prediction over heterogeneous graph triplets of the form (h, r, t) . Before prediction, the final meta-path representation obtained from the inter-meta-path attention mechanism is added to the head embedding:

$$\mathbf{h}' = \mathbf{h} + \mathbf{m}_{final}$$

where: $\mathbf{h}', \mathbf{h}, \mathbf{m}_{final} \in \mathbb{R}^{128}$. The head, relation, and tail embeddings are then concatenated into a single input vector:

$$\mathbf{x} = [\mathbf{h}'; \mathbf{r}; \mathbf{t}]$$

where: $\mathbf{x} \in \mathbb{R}^{384}$. The concatenated vector is passed through a multilayer linear Network consisting of:

- a linear layer $384 \rightarrow 128$,
- a ReLU activation function,
- and a final linear layer $128 \rightarrow 1$.

Formally:

$$\mathbf{z} = \text{ReLU}(W_1 \mathbf{x} + b_1)$$

$$s(h, r, t) = W_2 \mathbf{z} + b_2$$

where: $\mathbf{z} \in \mathbb{R}^{128}$ and: $s(h, r, t) \in \mathbb{R}$ is the predicted score of the triplet. Higher scores indicate a higher likelihood that the corresponding edge exists in the biomedical knowledge graph.

Model Training

For each positive triplet (h, r, t) a negative triplet was generated by replacing the tail node with a randomly sampled node of the same tail type (h, r, t^-) . Negative tail nodes were sampled uniformly from nodes of the same type without explicitly checking whether the resulting triplet already exists in the graph, since such verification can be computationally expensive in large graphs and the probability of randomly sampling an existing positive edge is relatively low.

The model computes a positive score $s^+ = s(h, r, t)$ and a negative score $s^- = s(h, r, t^-)$. We encourage the model not only to assign higher scores to positive triplets than to negative triplets, but also to maintain a minimum separation margin between them $s^+ - s^- \geq \gamma$ where γ is a margin hyperparameter controlling the required separation between positive and negative samples. In this project, the margin value was

set to $\gamma = 1$ thereby encouraging the model to learn a clearer distinction between valid and invalid triplets in the embedding space.

Training was performed using a margin ranking loss:

$$\mathcal{L} = \max(0, \gamma - s^+ + s^-)$$

This objective encourages positive triplets to obtain higher scores than negative triplets by at least the margin value.

The dataset is divided into training, validation, and test triplets:

- Training triplets: 4,762,678
- Validation triplets: 162,886
- Test triplets: 162,870

Four training configurations with different hyperparameter settings were evaluated:

Experiment	Epochs	Steps per Epoch	Batch Size	Learning Rate	Margin
small	5	100	32	3e-4	1.0
base	10	200	64	3e-4	1.0
longer	15	200	64	3e-4	1.0
Large batch	10	200	128	3e-4	1.0

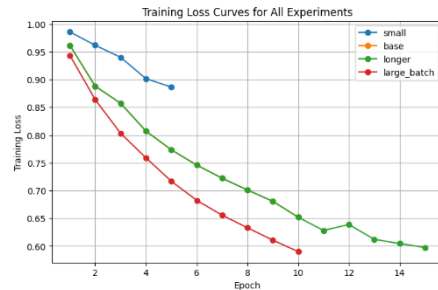
The validation set was used to evaluate the 4 model performance on unseen data. The test set was used to evaluate the best validated model performance.

Results

Training and Validation Results

The training loss consistently decreased across all experimental configurations, indicating successful optimization and stable learning behavior.

Increasing the number of training epochs led to further reductions in loss, while larger batch sizes resulted in faster convergence and lower final loss values. The large_batch configuration achieved the lowest training loss after 10 epochs, whereas the longer configuration continued improving with additional training epochs, despite minor fluctuations in later stages.



Model performance was evaluated using several standard classification metrics. ROC-AUC was used to measure the model's ability to distinguish between positive and negative triplets across different decision thresholds. Accuracy measured the overall fraction of correctly classified triplets. Precision evaluated the proportion of predicted positive triplets that were truly positive, while Recall measured the proportion of true positive triplets successfully identified by the model. Together, these metrics provide a comprehensive evaluation of the link prediction performance from multiple perspectives.

The experimental results demonstrate that increasing the training duration and batch size generally improved the link prediction performance. **The large_batch configuration achieved the best overall results**, obtaining the highest ROC-AUC value of 0.705. While the small configuration achieved the highest Recall, it also showed lower

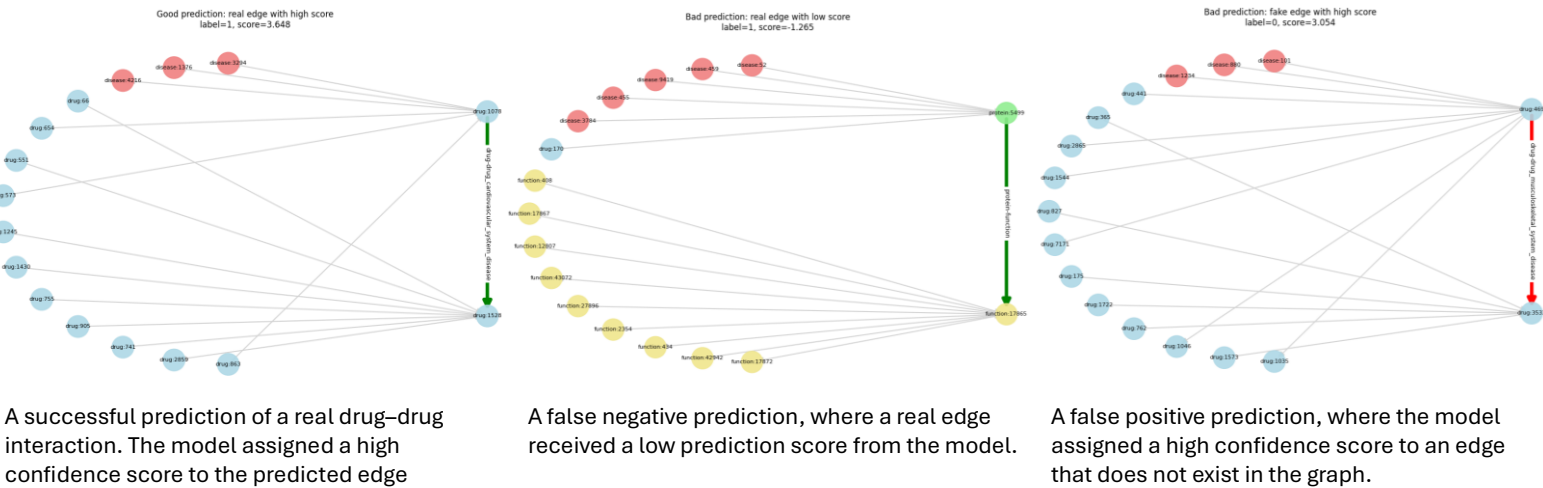
precision and ROC-AUC values, indicating weaker discrimination between positive and negative triplets. Overall, the results suggest that longer training and larger batch sizes contributed to more stable and accurate representations in the embedding space.

Experiment	Epochs	Steps per Epoch	Batch Size	Margin	Learning Rate	ROC-AUC	Accuracy	Precision	Recall
large_batch	10	200	128	1.0	0.0003	0.7054	0.5895	0.5594	0.843
longer	15	200	64	1.0	0.0003	0.6880	0.5800	0.5538	0.824
base	10	200	64	1.0	0.0003	0.6654	0.5855	0.5603	0.794
small	5	100	32	1.0	0.0003	0.5771	0.5175	0.5097	0.920

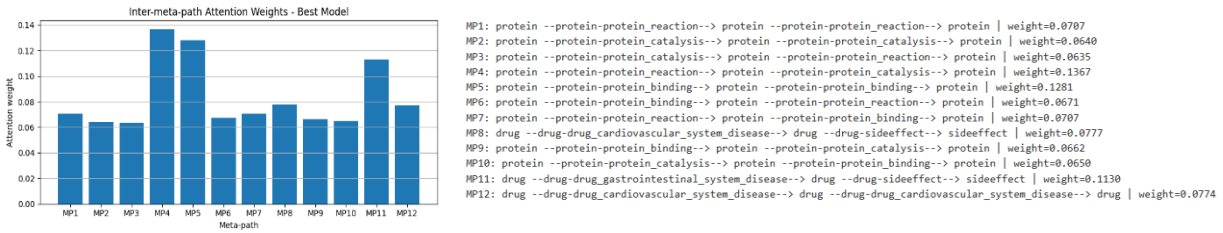
Best Model Analysis

The selected large_batch model was finally evaluated on the test set. The model achieved a test ROC-AUC of 0.7114, slightly higher than its validation ROC-AUC, indicating consistent generalization. The model also achieved high recall (0.8330), suggesting that it successfully identified most positive triplets, although the moderate precision (0.5745) indicates that some negative triplets were still classified as positive.

To better understand the model’s behavior beyond quantitative evaluation metrics, several local graph visualizations were generated for correctly and incorrectly classified triplets. Each visualization highlights the predicted edge together with its surrounding neighborhood in the biomedical knowledge graph.



Learned inter-meta-path attention weights were examined in order to identify which meta-path patterns received the highest importance during prediction.



Meta-paths involving repeated protein interactions and drug–side effect relationships received the highest attention weights, suggesting that these patterns contributed more strongly to the model’s final predictions compared to other meta-path structures.